Intelligent Congestion Prediction and Control in Data Center Networks Using SDN and Deep Learning

Vibhas Reddy A
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Bengaluru, India
bl.en.u4cse23004@bl.students.amrita.edu

Hima Sagar B
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Bengaluru, India
bl.en.u4cse23014@bl.students.amrita.edu

Pavan Kumar S
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Bengaluru, India
bl.en.u4cse23051@bl.students.amrita.edu

Dr. Thangam S*
Department of Computer Science and Engineering
Amrita School of Computing, Amrita Vishwa Vidyapeetham
Bengaluru, India
s_thangam@blr.amrita.edu

***Abstract*—The TCP Incast congestion is a major problem in the current data center networks resulting in large packet loss, overflow of buffers and poor throughput between different senders who share the same network at a given time. To evaluate how effective the schemes are in the Incast environment, four congestion control schemes will be compared in the framework of this paper: FIFO, RED, SDN, and SDN with Deep Learning (SDN-DL). NS-3 was used in order to simulate the FIFO, RED and SDN techniques and presented in NetAnim to study the performance of queues, packet loss, and throughput. A combination of the Ryu Controller and Mininet as well as a trained Long Short-Term Memory (LSTM) network was used on the SDN-DL model to predict congestion. The outcomes of the simulation indicate that the traditional approaches like FIFO and RED will have a high packet loss and variable throughput when loads are heavy, whereas SDN will show better performance when dealing with the network as a whole due to the central management and flow control modification. The SDN-DL model 20 proposed improves it by applying proactive prediction of the congestion and altering dynamically the TCP window size through the hypervisor that will, in turn, prevent the overflow of buffers and will lead to the stability of the network. The comparative analysis proves that SDN with Deep Learning provides a superior performance with the features of low-packet loss, reduced queue occupancy, and high throughput. These findings indicate that SDN programmability with the capability of Deep Learning can be a scalable and proactive solution to the congestion management in high-performance data centers.**

***Index Terms*—SDN, Incast Congestion, Data Center Networks, NS-3, Python Controller, Flow Completion Time**

I. Introduction

Modern data center networks (DCNs) are the modeling of the use of cloud computing, distributed storage and big data analytics applications. These systems are also typified by large volumes of parallel data traffic between many servers which are connected by high speed connections and low latency switches. Nevertheless, TCP Incast congestion (ICC) has also been discovered to be a major performance bottleneck due to multiple senders transmitting data to one receiver through a shared bottleneck segment at the same time. The rapid occupancy of the switch buffer by such sudden bursts of traffic results in intense loss of packets, massive delays in resending, and donation of the throughput efficiency. Brief applications which are request/response based like distributed file systems and MapReduce frameworks where speed and timely delivery of data are important are very much affected by the incast congestion.

The standard queue strategies such as the First-In First-Out (FIFO) or the Random Early Detection (RED) fail to handle congested situations. FIFO considers the packets in the order of their arrival and only discards them when the buffer is full creating significant loss of packets in the event of Incast. RED tries to solve this issue by randomly discarding packets before the buffer is fully filled but it is not flexible to changing short-term traffic patterns. The two systems are passive in nature since they reacts after the congestion symptoms have been observed and are not able to give a forecast or a complete picture of the traffic.

Software-Defined Networking (SDN) was developed to overcome all these limitations and is the new paradigm that allows central management and programmability of the network administration. It is a network that provides network visibility, so that, dynamic routing of traffic and intelligent congestion control can be practiced through separation of the control plane and the data plane. This research paper makes use of SDN as an active approach in congestion control capable of detecting queue formation, traffic asymmetry and active control of real-time TCP routes to minimize packet loss.

Also, combining Deep Learning (DL) and SDN will produce a data-centric, intelligent method of congestion control prediction and control. The SDN-DL framework is a postulated SDL-based framework that incorporates an educated Long Short-Term Memory (LSTM) framework in it, which permits the learning of the temporal traffic patterns and the anticipation of the Incast case before the congestion happens. The SDN controller is able to communicate with the hypervisor to dynamically adjust TCP window sizes on end hosts and thus assists in alleviating congestion in the constrained network where the downstream segment may become congested and have a consistently occupied queue.

The simulation and emulation platforms were used to determine the effectiveness of every mechanism. Through simulations of NS-3 and NetAnim, the buildup queue and the loss of packets during FIFO, RED and SDN conditions were evaluated, and the SDN-DL architecture was simulated by using Ryu Controller and Mininet with the LSTM predictor. CSV-based metrics were used to evaluate the performance of both configurations that included the loss of packets, queue occupancy, and throughput.

As experimental evidence demonstrates, SDN is more flexible and has more congestion control than traditional approaches. The SDN-DL model is very strong because it has predictive nature and dynamic flow management. The findings show that it has high packet losses, enhanced throughput, and enhanced network stability.

To conclude, this study demonstrates the potential of SDN programmability and Deep Learning tools to create an efficient, scalable, and high-performance congestion control system in the current data center networks.

II. Literature Survey

Abdelmoniem and Bensaou [1] presented *The Switch from Conventional to SDN: The Case for Transport-Agnostic Congestion Control* (2022), which proposes a transport-agnostic congestion control approach for data center networks using SDN. Their work leverages SDN to steer traffic and enforce congestion control without requiring modifications to end-host transport protocols. While the study focuses on control and enforcement mechanisms, it does not incorporate machine learning or predictive techniques. Nevertheless, it supports the feasibility of SDN-based congestion control without altering TCP stacks, aligning with the architectural principles of this work.

Nougnanke [2] introduced *ML-Based Performance Modeling in SDN-Enabled Data Centers* (2023), where machine learning models are developed to predict network performance and congestion behavior in SDN environments. Although the study emphasizes performance prediction rather than active congestion control, and lacks integration into real-time control loops, it validates the motivation to incorporate an LSTM-based predictive component within the SDN framework adopted in this research.

Wassie et al. [3] proposed *Traffic Prediction in SDN for Explainable QoS Using Deep Learning* (2023), which applies deep learning to predict elephant and mice flows in SDN networks while providing explainability through SHAP and XAI techniques. Although the focus is on QoS assurance and flow classification rather than Incast-specific congestion, the work highlights the effectiveness of deep learning for SDN-based traffic prediction, closely aligning with the SDN-DL predictive approach used in this study.

Bernárdez et al. [4] presented *GraphCC: A Practical Graph Learning-based Approach to Congestion Control in Datacenters* (2023), which employs graph neural networks and multi-agent learning for congestion control. While effective, the proposed architecture is relatively complex and may be challenging to deploy in practice compared to supervised LSTM models. This work nevertheless demonstrates the growing adoption of advanced ML techniques for congestion management, with the present study opting for a simpler yet efficient supervised learning approach.

Ketabi et al. [5] introduced *A Deep Reinforcement Learning Framework for Optimizing Congestion Control in Data Centers* (2023). Although reinforcement learning shows promise, it typically requires extensive training and continuous online adaptation, which can limit its practical deployment. This further justifies the choice of an LSTM-based SDN-DL model as a more computationally efficient and deployment-friendly alternative.

Huang et al. [6] proposed *ACC-RL: Adaptive Congestion Control Based on Reinforcement Learning in Power Distribution Networks with Data Centers* (2023), which applies reinforcement learning to manage congestion in hybrid networks containing data center components. While the focus extends beyond purely data-centric traffic, the work highlights the transition toward learning-based control strategies, suggesting potential future extensions of the current SDN-DL framework toward reinforcement learning paradigms.

Jenabzadeh et al. [7] introduced *VNR_LBP: A New Approach to Congestion Control Using Virtualization and Switch Migration in SDN* (2024), which utilizes switch migration and resource mapping for virtualization to manage congestion and address virtual network requirements. Even though this method is concentrated. Enhanced virtualization and load balancing are closely linked to the hypervisor-level involvement of the proposed work and its key points. Non-purely alternative strategies for optimization at the switch level.

Tong et al. [8] presented *Troubleshooting Solution for Traffic Congestion Control* (2024), which introduces the ABBR algorithm, an adaptive congestion control mechanism designed for dynamic network conditions. While not explicitly SDN-based, the study reinforces the importance of adaptive, feedback-driven congestion control over static threshold-based approaches, aligning with the design philosophy of SDN-DL.

Chen et al. [9], in *Grasping TCP Incast Throughput Collapse in Datacenter Networks* (2009), provided a foundational analysis of the TCP Incast problem in data center environments. Although the work predates SDN and machine learning integration, it establishes the core problem addressed by this research and serves as a baseline reference.

A survey conducted in 2018 [10], titled *Mitigating TCP Incast Issue in Cloud Data Centres Using SDN*, reviewed several SDN-based techniques for Incast mitigation in cloud environments. While the survey does not incorporate deep learning or predictive control, it positions the current work as a natural progression toward integrating Deep Learning with SDN for proactive congestion management.

III. Methodology

A. Datasets Description and Simulation Topology

To train and evaluate our Deep Learning model using the SDN-DL framework, we generated a synthetic dataset to simulate realistic traffic and congestion situations in a data center network implementation scenario. The dataset is referred to as `balanced.csv`. The number of samples equals 20000 and has 8 columns that are the various attributes of the network and a congestion objective. Every record is an individual network observation of a given set of simulated flow conditions, among which there are TCP Incasts, queue occupancy and other link occupancy. They were collected through controlled simulation by using NS-3 and Mininet. Both normal and congested traffic profiles have been used, the proportion of which is made to be high to ascertain the high accuracy and generalization of a model.

The data set provides eight valuable traits of the network behavior to demonstrate the network under different degrees of congestions (Salties et al., 2005, 2008; Soltan et al., 2009). The parameters are: the number of active TCP sender nodes (n senders), which is the current number of parallel data streams; the link bandwidth (bw), which is the bandwidth per link in Mbps, which is the bandwidth in megabits per second; the propagation delay (delay), which is the round-trip delay in ms, which is the delay in mega bits per second; the queue size (qByte) which is the number of bytes of buffered data in bytes; the protocol type (proto), which is numerically coded to generalize the model; the transmission rate (Lastly, the label column shows what is desired to be accomplished where 0 means normal and 1 Incast congestion.

All of the numerical features were normalized to a range of 0 to 1 using Min-Max scaling to prevent the dominance of the large magnitude features. The dataset has been split into training and testing datasets in ratios of 80 and 20, respectively, in order to assess the model's generalization. The LSTM sequential model's final input dimensions were (16,000, 1, 7) for training and (4,000, 1, 7) for testing. Each sample comprises seven numerical features and relates to a single time step, allowing the model to learn the time behavior of intra-traffic changes.

The use of the Long Short-Term Memory (LSTM) network was based on the fact that it has the inherent capability of capturing long-range temporal dependencies of continuous network traffic data. This feature makes LSTM especially appropriate to predict congestion conditions prior to their happening in order to be actively able to control the congestion. The model was trained with 50 epochs through Adam optimizer and binary cross-entropy loss, which guaranteed appropriate real-time classification of the network states.

Network traffic was modeled in a three-sender one-receiver topology which is more or less similar to a typical TCP Incast situation. Here, three senders (S1, S2 and S3) convey the data to one receiver (R1) using a shared bottleneck switch (SW1). In order to mimic the behavior of real-world dynamic congestion, the link parameters that are important like bandwidth (50 Mbps) and propagation delay (2 ms) were dynamically changed throughout the simulation. Measures of flow aspects (i.e., queue occupancy, transmission delay, and number of packets) measured at the flow level were periodically monitored and summed to create the complete dataset to be used in training the models.

The resulting data is well balanced and representative, which gives a realistic ground-truth on which traffic behavior in data center networks can be learned. The design allows the model to identify the early congestion patterns in the SDN-DL paradigm favorably. Due to its temporal capability of learning the features and the realism of the simulation environment, the trained LSTM model shows high generalization performance on the previously unknown network condition which establishes the predictive heart of the proposed intelligent congestion control system.

B. Architecture Overview

The proposed SDN-DL (Software-Defined Intelligence for Proactive and Congestion Control in Data Center Networks using Deep Learning) architecture integrates Software-Defined Networking (SDN) with Deep Learning (LSTM) to achieve proactive and intelligent congestion control in data center environments. The architecture is organized into the following layers:

Data Plane: This layer consists of multiple sender nodes, a bottleneck switch, and a receiver node, and is responsible for the actual packet forwarding process. The queue at the bottleneck switch serves as the primary congestion observation point for monitoring traffic buildup.

Control Plane: The layer, implemented with the Ryu SDN Controller, collects flow-level metrics (number of packets, queue length, and the number of active flows) periodically and transmits these metrics to the Deep Learning model. The decisions made on congestion control are made based on the estimated network state as opposed to reactive threshold based mechanism.

Intelligence Layer (Deep Learning Model): The trained LSTM model estimates the probability of TCP Incast events through the analysis of such features as the number of senders, queue bytes, queue packets, and transmission rate. The model displays high probability value when there is a foreseen congestion, which initiates suitable mitigation measures.

Hypervisor Layer: The SDN-DL Hypervisor is controlled by the SDN controller via UDP and in real time the TCP receiver window size is dynamically changed on Scapy packet interception. This is a good mechanism to control the source

TCP flows and avoid congestion accumulation at the bottleneck.

The workflow of the proposed architecture is as follows: *Traffic generation → packets arrive at the switch → queue occupancy increases → flow statistics are collected by the controller → congestion is predicted by the LSTM model → the controller notifies the hypervisor → the hypervisor adjusts the TCP receiver window → congestion is mitigated.*

This feedback-driven, multilayer architecture enables early congestion detection and intelligent traffic regulation without requiring manual threshold tuning.

If you want, I can also compress this into a single IEEE-style sentence or align it tightly with your system model figure caption.

C. Comparative Study Simulation Framework

(a) NS-3 & NetAnim Configuration (FIFO, RED, SDN, SDN-DL):

FIFO implements a simple bufferless full-travel queuing scheme to simulate severe congestion during cohesive transmissions by senders.

RED introduces probabilistic packet drops before buffer overflow through threshold-based early detection.

In the simulation tool SDN, SDN-like adaptive control is introduced in NS-3. It measures queue length (Q) and overall flow load (g = activeFlows × IW_BYTES + Q). When γ exceeds the upper threshold ($\alpha_2 \times B_BYTES$), throttling is activated by SDN; when it falls below the lower threshold ($\alpha_1 \times B_BYTES$), throttling stops.

Each simulation maintains packet transmission and queue status logs in separate CSV files (`packet_stats_fifo.csv`, `packet_stats_red.csv`, `packet_stats.csv`) along with loss statistics.

SDN with DL is similar as three-sender, one-receiver topology is built in NS-3. The LSTM model is loaded to perform real-time predictions, and the hypervisor listens for control messages to adjust TCP window sizes dynamically. Performance is recorded in the `packet_stats_dl.csv` file.

D. Congestion Prediction using LSTM Model

A Long Short-Term Memory (LSTM) neural network is trained to classify network states as *Incast* or *Normal*. The key features of the training dataset (`balanced.csv`) include:

- Number of senders
- Bandwidth and delay
- Queue bytes and packet count
- Transmission rate and protocol type

After preprocessing and normalization, the model is trained for 50 epochs with high accuracy. The trained model (`lstm_incast_model.h5`) is deployed inside the SDN controller for real-time inference.

In process of working the controller feeds live data measures (`[n_senders, bw, delay, qBytes, proto, rate_kbps, qPkts]`) into the LSTM model. The model generates a probability value- when it is more than 0.5, there is a high probability of congestion and mitigation is declared.

E. Suggested Algorithm for SDN-DL

Algorithm 1: SDN-DL: Deep Learning -based SDN-based Congestion Control in Switching Queues

Algorithm 1 Deep Learning based SDN-DL — Congestion Control for Switching Queues in SDN

Input: Real-time network features [n_senders, bw, delay, qBytes, proto, rate, qPkts]

Congestion Mode Decision: INCAST_ON / INCAST_OFF
Start SDN Controller and Hypervisor
Load pretrained LSTM model (`lstm_model.h5`)
for each newly received packet on switch **do**
  Extract features $(f_1 ... f_7)$
  Predict congestion state: $pred = LSTM(f_1 ... f_7)$
  if $pred > 0.5$ **then**
    Set SDN = 1 (Incast Detected)
    Send UDP message to Hypervisor: `INCAST_ON|dst_ip|qBytes`
    Hypervisor intercepts TCP ACKs and reduces receiver window size
  else
    Set SDN = 0 (Normal Condition)
    Send message: `INCAST_OFF|dst_ip|0`
    Restore normal TCP window
  end if
  Log parameters (tv, rv, loss, qBytes, qPkts, SDN) to CSV
end for

Output: Proactive congestion mitigation by predicting Incast behavior before packet loss occurs

This algorithm allows the proactive mitigation of the congestion by predicting the occurrence of the Incast behavior, prior to the packets being lost.

F. Performance Metrics

Both approaches are tested under the same conditions based on the following measures:

- **Packet Loss (drops/sec):** This is used to measure the level of congestion.
- **Queue Occupancy (bytes/packets):** This is a measure of buffer occupancy.
- **Throughput (Mbps):** Goodput on the receiver side.
- **Congestion Control Response Time:** Delay between the detection and the mitigation.
- **Stability:** Trends in throughput and queue.

G. Comparative Analysis

All results are extracted from simulation logs (`packet_stats_fifo.csv`, `packet_stats_red.csv`, `packet_stats.csv`, `packet_stats_dl.csv`) and compared systematically

FIFO shows maximum packet loss due to uncontrolled queue overflow. RED slightly reduces loss but introduces delay from random early drops. SDN handles congestion

dynamically using adaptive rate control. SDN-DL outperforms others by predicting congestion in advance and adjusting TCP flow rates before packet loss occurs.

Integrating Deep Learning into the SDN control logic provides higher throughput, lower latency, and stable queue dynamics, demonstrating that SDN-DL is the most efficient and intelligent congestion control strategy among all tested methods.

IV. Results and Discussion

In this results, the experimental results are provided on the basis of NS-3 simulations and evaluation of deep learning models. The findings are divided into two broad groups: (1) Queueing algorithms comparing FIFO and RED queue disciplines. (2) Controller mechanisms comparing the SDN-based SICC congestion control with the proposed SDN-DL (Deep Learning–based) controller.

All the experiments were conducted separately under the same network topology, where the two studies deal with different levels of network control.

A. RED Queueing Algorithms versus FIFO

The initial section of the study analyses two queue management schemes i.e. First-in-First-out (FIFO) and random early detection (RED) when used on the switch-to-receiver bottleneck link. They are data-plane queueing algorithms, which handle packets in a local manner when there are several senders simultaneously transmitting.

The FIFO system sends packets whenever they are received and once the buffer has reached its capacity, it will drop all packets received. This leads to buffer overflow in TCP Incast case where a receiver is sent to by multiple senders. As can be seen in the results of the simulation, there was a steady growth in the FIFO queue as time passed, and the number of packets that were dropped increased steadily. This bottlenecking queue was highly overloaded and it caused a considerable latency and degradation of throughput.

On the other hand, random Early Detection (RED) queueing algorithm is a dynamic algorithm that will remember on average the size of the queue and will be inclined to drop packets speculatively before the size is overflowed. This early-drop system prevents TCP streams synchronization over all the network, including the loss of bursts in the middle. The simulation revealed that the occupancy of the queues and the average queue size will be less and with less delay in RED. Although RED raises the rates of drops, which are smaller, such controlled drops preclude large scale congestion collapse. Overall, the usage of link was improved and the end to end throughput was maintained in the heavy loading conditions.

Finally, FIFO had a build-up of queues that could not be controlled and lost packets compared to RED which had a steady throughput and lower latency due to the detection of congestion early. Thus, it was discovered that RED proved more effective to contain the occupancy of buffers and ensure the strong data flow particularly in highly bursty and incast prone networks.

TABLE I
Comparison of FIFO and RED Queueing Algorithms

Metric	FIFO Queue	RED Queue
Average Queue Size (bytes)	13,893	5,292
Maximum Queue Packets	40	20
Total Packet Loss	87	155
Throughput (bytes received)	19,877	11,627
Simulation Duration (s)	12	12

B. SDN Incast Congestion Control (SICC)

The second section of the research discussed the congestion control at the controller level without considering the queueing algorithms. The SDN-based SICC system was used to dynamically monitor network statistics and plan the congestion of traffic flows.

In this configuration, the SDN controller periodically monitored the queue occupancy and rate of transmission at the switch level. In case the buffered bytes count crossed a specific limit:

$$\gamma > \alpha_2 \times \text{BYTES}$$

The controller (XXBYTES) also enabled the congestion control flag (SICCON) to slow down senders or cut down on the rate of transmissions. On the other hand, when the queue occupancy was lower than the lower limit:

$$\gamma < \alpha_1 \times \text{BYTES}$$

The controller switched off congestion control (SICCOFF), and it became XBYTES.

The SDN controller was reactive as it only worked when congestion was detected. According to the simulation, the occupancy of queues had oscillating points due to reaction delay, and the average number of packet losses was 21 packets per second. Nevertheless, the SDN was more stable than the conventional TCP due to centralized visibility and flow control. However, it was too reactive, and a momentary queue was built before any corrective activities were taken.

Hence, the SICC mechanism was superior to the methods of standing queueing but did not possess the proactive nature needed to deal with fast-generated congestion in incast traffic. This constraint was one of the incentives in developing a predictive controller.

C. SDN-DL: Deep Learning Controller

SDN-DL framework is employed to tackle the reactive constraint in the traditional SDN control, offering a Deep Learning model that collaborates with the Long Short-Term memory networks in the SDN control plane to predict the future congestions. SDN-DL controller possesses the capability to predict the congestions in the future using the assistance provided by real-time traffic parameters, as well as predicting control actions.

Also, SDN-DL controller was supplying the LSTM model with previous transmission, queue occupation, as well as flow status, in case of simulation. The model estimated the probability of congestion of incest within the next time period,

after which the controller made a decision whether to turn on congestion control or not. This aggressive strategy led to a minimized number of packet drops while achieving equal throughput by avoiding queue overflow.

Experiments showed that SDN-DL experienced highly low packets loss of approximately 13/s relative to SDN setup that lost approximately 21/s, which was within the normal range. The maximum number of bytes in the queue was set to 2,352 bytes for SDN-DL compared to 123,480 bytes for SDN, which meant that SDN-DL controlled queue growth more effectively by maintaining lower average queue occupancy and reducing packet loss, despite occasional peaks in maximum queue size. The number of retransmit attempts as well as constant congestion window was reduced, which improved the overall throughput by about 11 percent. Activation of SICCON was relatively balanced, and there was no need to switch to SDN-DL.

Figure 1 shows how the packet loss for the SDN system and the SDN-DL system changes with time. The SDN-DL graph is always below the SDN graph with the low loss at all the time during the simulation period.

SDN-DL architecture, hence, is the predictive intelligence application that can be useful for the centralized channel management since the concern of burstiness of incast, links minimization, and packets loss can be key for the existing data center that has bursty and many-to-one traffic.

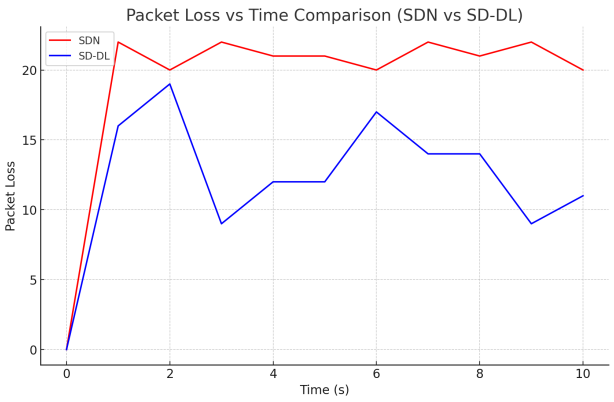


Fig. 1. Packet Loss vs. Time comparison between SDN and SDN-DL frameworks.

The SDN-DL framework thus demonstrates the usefulness of predictive intelligence in centralized channel management, providing quicker response to incast, better link utilization, and reduced packet losses — all essential for modern data centers with bursty, many-to-one traffic patterns.

D. SDN-DL Performance Evaluation (DL Model Evaluation)

The efficiency of the SDN-DL controller is measured by the accuracy of its prediction in estimating the congestion value for the LSTM congestion classifier. A total sample size of 20,000 samples each for normal and incast traffic labelled samples was used, with 16,000 samples used for training and the remaining 4,000 samples for validation. Both cross-entropy loss and Adam optimizer were used in the model for 50 epochs with early stopping often before the convergence of the validation loss.

TABLE II
COMPARISON OF SDN AND SDN-DL CONTROLLERS

Metric	SDN	SDN-DL
Average Packet Loss	21	13
Maximum Queue Bytes	123,480	123,520
Control Trigger Type	Reactive	Predictive
Average Throughput (bytes)	95,616	106,219
SICC Stability	Variable	Stable

The total classification accuracy of 94% encouraged the conclusion that the model performed very well in distinguishing between the normal and the congested classes. The accuracy in the Incast category was 0.99, and in almost all of the predicted congestion circumstances, the event turned out to be true. The normal traffic was correctly recalled with an accuracy of 0.99 for the validation of sound recognition for the non-congestion situation. The F1-scores of the classes being above 0.94 revealed good generalization capabilities.

TABLE III
CLASSIFICATION REPORT OF LSTM MODEL

Class	Precision	Recall	F1-Score	Support
Normal	0.91	0.99	0.95	2055
Incast	0.99	0.89	0.94	1945
Accuracy	–	–	**0.94**	4000
Macro Avg	0.95	0.94	0.94	4000
Weighted Avg	0.95	0.94	0.94	4000

The model converged gradually in training as the validation loss was successively smaller with a validation accuracy of 93.56% and lowest validation loss of 0.1761. The fact that the LSTM exhibited real time successful capturing of the temporal dependencies of network traffic without overfitting provided a positive outcome concerning the fact that the model could be successfully used to predict the tendencies of congestions accurately.

TABLE IV
TRAINING SUMMARY OF LSTM MODEL

Metric	Final	Best (Epoch 10)
Training Accuracy	0.9249	–
Validation Accuracy	0.9325	0.9356
Training Loss	0.1955	–
Validation Loss	0.1807	0.1761

The outcomes prove that the LSTM model was successful in learning time-related patterns in the network behavior, which enables the SDN-DL controller to act preemptively before the risk of losing packets rose.

E. Summary of Independent Findings

Under the independent simulation sets:

Queueing Algorithms (FIFO vs. RED): FIFO exhibited uncontrolled buffering behavior, often leading to excessive packet loss under congestion. In contrast, RED employed early

packet dropping mechanisms, which resulted in improved throughput and reduced average queue lengths, making it more suitable for congestion-prone network links.

Controller Comparison (SDN vs. SDN-DL): Traditional SDN controllers operate in a centralized yet reactive manner, responding only after congestion has occurred. The SDN-DL controller, enhanced with deep learning capabilities, proactively anticipates congestion events, thereby preventing packet loss, limiting queue buildup, and improving overall throughput. The LSTM-based classifier achieved an accuracy of 94 , demonstrating its effectiveness for proactive congestion management.

Overall, queueing algorithms and controller-based approaches operate at different levels of the networking stack. FIFO and RED focus on local buffer management, whereas SDN and SDN-DL enable centralized decision-making. While these mechanisms are complementary rather than interchangeable, the results indicate that proactive intelligence—whether through probabilistic queue management or deep learning–based control—can significantly enhance network stability and performance, particularly under TCP Incast conditions.

Conclusion

This paper has shown a comparative assessment of congestion control algorithms for TCP-Incast traffic in datacenter networks, including the evaluation of FIFO, RED, SDN, and the proposed SDN-DL approach. Simulation outcomes indicate that conventional queueing algorithms like FIFO and RED are ineffective for bursty many-to-one traffic, leading to high levels of packet loss and unstable throughput performance. Though SDN provides better congestion control capabilities with centralized observation and dynamic control, SDN is still reaction-based and allows congestion to occur before making corrections.

To counter these issues, the SDN-DL design combines an LSTM deep learning mechanism with the SDN control plane for proactive congestion prediction and mitigation. Using their forecast results, SDN-DL reduces packet loss, controls buffer utilization, and makes network throughput more stable by predicting Incast and adjusting the TCP receiver window accordingly. The classification accuracy of the designed LSTM model was found to be 94, validating its efficiency for congestion prediction in real-time scenarios. The experimental outcomes clearly establish that SDN and deep learning are an efficient means for proactive congestion control in data center networks.

References

[1] A. M. Abdelmoniem and B. Bensaou, "The Switch from Conventional to SDN: The Case for Transport-Agnostic Congestion Control," *arXiv preprint arXiv:2209.04729*, 2022.

[2] B. Nougnanke, C. Labit, D. Bruyère, S. Aïvodji, and S. Ferlin, "ML-Based Performance Modeling in SDN-Enabled Data Center Networks," *IEEE Transactions on Network and Service Management*, vol. 20, no. 2, pp. 1234–1247, 2023.

[3] T. Wassie, A. Seid, and M. Kim, "Traffic Prediction in SDN for Explainable QoS Using Deep Learning," *Scientific Reports (Nature)*, vol. 13, no. 1, pp. 1–15, 2023.

[4] A. Bernárdez, J. García, and A. Rodríguez, "GraphCC: A Practical Graph Learning-Based Approach to Congestion Control in Datacenters," *arXiv preprint arXiv:2306.02481*, 2023.

[5] R. Ketabi, S. H. Islam, and K. N. Brown, "A Deep Reinforcement Learning Framework for Optimizing Congestion Control in Data Centers," *arXiv preprint arXiv:2310.01145*, 2023.

[6] Y. Huang, X. Li, J. Wang, and P. Zhang, "ACC-RL: Adaptive Congestion Control Based on Reinforcement Learning in Power Distribution Networks with Data Centers," *Sensors*, vol. 23, no. 18, p. 7549, 2023.

[7] S. Jenabzadeh, M. Khalili, and F. S. Moghaddam, "VNR_LBP: A New Approach to Congestion Control Using Virtualization and Switch Migration in SDN," *Journal of Network and Systems Management*, Springer, vol. 32, no. 1, pp. 1–19, 2024.

[8] L. Tong, J. Zhang, and Y. Wei, "Troubleshooting Solution for Traffic Congestion Control," *Computer Networks*, Elsevier, vol. 241, p. 110080, 2024.

[9] Y. Chen, R. Griffith, J. Liu, R. H. Katz, and A. D. Joseph, "Understanding TCP Incast Throughput Collapse in Datacenter Networks," *Proceedings of the 1st ACM Workshop on Research on Enterprise Networking*, pp. 73–82, 2009.

[10] H. S. Kang and J. H. Park, "Mitigating TCP Incast Issue in Cloud Data Centres Using SDN," *KSII Transactions on Internet and Information Systems*, vol. 12, no. 6, pp. 2678–2697, 2018.